RoBe Mobile App

Backend Application Development Guide

FastAPI V1 technical documentation for architecture, service boundaries, project structure, and AI-assisted implementation workflow

Document Purpose

• Defines the recommended backend architecture for the RoBe AI chat application using FastAPI, PostgreSQL, Redis, Celery, and AWS-managed services.

• Provides a project structure and implementation conventions that AI coding tools such as Cursor and Claude Code can follow consistently.

• Establishes standards for API design, data access, background jobs, integrations, testing, observability, and release readiness for V1.

V1 backend scope covered by this guide

• Phone-based authentication support, user accounts, and secure token/session patterns.

• User profile, multi-pet profile, topic-driven chat, chat history, recommendations, notifications, and settings APIs.

• Operational foundations for background jobs, AI orchestration, product recommendation logic, and future multilingual or premium features.

• Modular monolith organization that is practical for a small team and compatible with AI-assisted code generation.

Prepared for implementation planning

1. Backend Technology Stack

RoBe V1 should use a modular but operationally simple backend stack. The main goal is to deliver stable chat APIs, reliable product recommendation workflows, and secure user and pet management without introducing early infrastructure complexity.

Layer	Recommended Technology	Why It Fits RoBe V1
API framework	FastAPI	High-performance Python framework with clean typing, async support, and strong developer productivity.
Language	Python 3.12	Strong ecosystem for AI services, backend integration, data validation, and automation.
ORM	SQLAlchemy 2.x	Modern typed ORM and query layer for PostgreSQL models and transactional workflows.
Schema validation	Pydantic v2	Strict request and response validation, config typing, and predictable serialization.
Database migrations	Alembic	Version-controlled schema changes and repeatable deployment-safe migrations.
Primary database	PostgreSQL	Core transactional store for users, pets, chats, topics, products, and recommendations.
Cache / broker	Redis	Supports caching, throttling, short-lived context, OTP/session helpers, and task brokering.
Async jobs	Celery	Background processing for notifications, AI summaries, recommendation tasks, and cleanup flows.
Containerization	Docker	Consistent local, staging, and production environments for API and worker services.
Hosting	AWS ECS Fargate	Container hosting without managing EC2 servers directly.
Managed DB	AWS RDS PostgreSQL	Managed PostgreSQL with backups, monitoring, snapshots, and scaling options.
Object storage	Amazon S3	Pet photos, exported files, generated assets, and optional attachment storage.
CDN	Amazon CloudFront	Fast delivery of public assets and media.
Monitoring	CloudWatch + Sentry	Operational logs, metrics, traces, crash visibility, and incident diagnosis.
CI/CD	GitHub Actions	Automated linting, testing, image builds, migration checks, and deployment workflows.
Infra as code	Terraform or AWS CDK	Repeatable and reviewable infrastructure setup for every environment.

1.1 Backend design principles

- Build a modular monolith first, not a distributed microservice system.
- Keep business rules in service layers instead of spreading them across routes, models, and tasks.
- Use strict typing for request payloads, response models, settings, repository interfaces, and task contracts.
- Separate transport logic, application logic, persistence logic, and external integrations clearly.
- Organize code so AI coding tools can safely add or modify modules without breaking hidden dependencies.

2. Backend architecture overview

The backend should follow a layered architecture inside a modular monolith. API routes should stay thin, services should own orchestration and business decisions, repositories should manage persistence, and task workers should handle asynchronous or non-blocking workloads.

Architecture Layer	Responsibility
API layer	FastAPI routers, dependency injection, request parsing, auth guards, pagination params, and response shaping.
Application / service layer	Core use cases such as onboarding, pet management, chat session orchestration, recommendation generation, and notification logic.
Domain model layer	SQLAlchemy entities, enums, domain rules, and invariants for users, pets, chat sessions, products, and recommendations.
Repository / data layer	Query abstraction, transactions, model persistence, read models, and optimized list/detail queries.
Task layer	Celery jobs for delayed, heavy, or retryable work such as AI summarization, notifications, and cleanup routines.
Integration layer	AWS services, AI providers, SMS/OTP services, push gateways, monitoring, and storage utilities.
Cross-cutting core	Settings, security, logging, error handling, app wiring, constants, and shared helpers.

2.1 Core request flow

- Client sends request to a versioned FastAPI endpoint.
- Router validates request models and resolves dependencies such as auth context, DB session, cache client, and current user.
- Service layer executes the use case, calling repositories and integrations as needed.
- Transactional work is committed once the service completes successfully.
- Heavy side effects are dispatched to Celery tasks or queued after commit.
- Response schemas map internal models into frontend-safe payloads with stable contract boundaries.

3. Recommended FastAPI project structure

The folder structure below is recommended for V1. It is detailed enough for a real product and structured enough that AI assistants can generate files in predictable locations.

```
robe-backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── deps/
│   │   ├── errors/
│   │   ├── middleware/
│   │   ├── routers/
│   │   │   ├── auth.py
│   │   │   ├── users.py
│   │   │   ├── pets.py
│   │   │   ├── topics.py
│   │   │   ├── chat.py
│   │   │   ├── history.py
│   │   │   ├── recommendations.py
│   │   │   ├── notifications.py
│   │   │   └── settings.py
│   │   └── v1.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── logging.py
│   │   ├── exceptions.py
│   │   ├── constants.py
│   │   └── telemetry.py
│   ├── db/
│   │   ├── base.py
│   │   ├── session.py
│   │   ├── enums.py
│   │   └── seed/
│   ├── models/
│   │   ├── user.py
│   │   ├── role.py
```

```
│   │   ├── pet.py
│   │   ├── topic.py
│   │   ├── ai_personality.py
│   │   ├── chat_session.py
│   │   ├── message.py
│   │   ├── product.py
│   │   ├── recommendation.py
│   │   └── notification.py
│   ├── schemas/
│   │   ├── common/
│   │   ├── auth/
│   │   ├── users/
│   │   ├── pets/
│   │   ├── chat/
│   │   ├── recommendations/
│   │   └── notifications/
│   ├── repositories/
│   │   ├── base.py
│   │   ├── user_repository.py
│   │   ├── pet_repository.py
│   │   ├── topic_repository.py
│   │   ├── chat_repository.py
│   │   ├── recommendation_repository.py
│   │   └── notification_repository.py
│   ├── services/
│   │   ├── auth/
│   │   ├── users/
│   │   ├── pets/
│   │   ├── chat/
│   │   ├── recommendations/
│   │   ├── notifications/
│   │   └── settings/
│   ├── tasks/
│   │   ├── celery_app.py
│   │   ├── chat_tasks.py
│   │   ├── recommendation_tasks.py
```

```
│   │   ├── notification_tasks.py
│   │   └── maintenance_tasks.py
│   ├── integrations/
│   │   ├── ai/
│   │   ├── aws/
│   │   ├── sms/
│   │   ├── push/
│   │   └── storage/
│   ├── cache/
│   │   ├── redis.py
│   │   ├── keys.py
│   │   └── throttling.py
│   ├── utils/
│   │   ├── dates.py
│   │   ├── pagination.py
│   │   ├── ids.py
│   │   └── text.py
│   └── tests/
│       ├── unit/
│       ├── integration/
│       ├── api/
│       └── fixtures/
├── alembic/
├── docker/
│   ├── api.Dockerfile
│   ├── worker.Dockerfile
│   └── docker-compose.yml
├── scripts/
│   ├── seed_topics.py
│   ├── seed_products.py
│   └── backfill_chat_summaries.py
├── .env.example
├── pyproject.toml
├── README.md
└── Makefile
```

Directory	Purpose
app/api	Versioned routes, dependency wiring, request/response handling, middleware, and API error mapping.
app/core	Central configuration, security helpers, logging, exception definitions, and shared backend constants.
app/db	Database setup, SQLAlchemy base, sessions, enums, and seed helpers.
app/models	ORM entities representing the relational data model.
app/schemas	Pydantic request and response contracts grouped by feature area.
app/repositories	Persistence layer and reusable query logic that should not live in services or routers.
app/services	Feature-specific business logic and orchestration for use cases.
app/tasks	Celery app configuration and asynchronous job definitions.
app/integrations	Wrappers for AI provider calls, AWS services, SMS/OTP, push, and object storage.
app/cache	Redis connection helpers, cache keys, throttling, and short-lived state utilities.
app/tests	Unit, integration, and route-level tests alongside fixtures.
alembic	Database migration source of truth.
docker	Container definitions and local multi-service development setup.
scripts	Operational scripts for seeding, maintenance, and controlled backfills.

3.1 Suggested chat service sub-structure

```
app/services/chat/
├── chat_service.py          # chat session lifecycle and orchestration
├── message_service.py       # message creation, validation, sequencing
├── session_service.py       # history list, archive, rename, bookmark-ready logic
├── prompt_builder.py        # builds AI payloads from user, pet, topic, and personality context
├── response_mapper.py       # maps AI provider output into message and recommendation records
├── recommendation_resolver.py# selects products based on chat context and business rules
└── summary_service.py       # session summary/title generation and maintenance
```

4. Core backend modules for RoBe V1

Module	Primary Responsibility	Notes
auth	Phone login, OTP verification, token issuance, token refresh, and account bootstrap.	Should support either Cognito-backed or custom JWT-backed flows behind a stable service interface.
users	User profile retrieval and update operations.	Keep profile APIs separate from auth lifecycle concerns.
pets	Multi-pet profile CRUD, pet selection context, and pet metadata validation.	Important because chat and recommendations may depend on pet context.
topics	Topic card listing and topic metadata exposure.	Should remain simple and cache-friendly.
chat	Session creation, message persistence, AI orchestration, summary generation, and inline recommendation emission.	The most critical module and the one that should receive the strongest service boundaries.
history	Session list, filters, archive status, pagination, and future search readiness.	Can share repositories with chat but expose its own read-focused endpoints.
recommendations	Product recommendation history, save/click feedback, and recommendation detail APIs.	Should convert AI output into user-facing product data cleanly.
notifications	Notification center APIs, read status updates, and delivery event handling.	Keeps the backend ready for push notification workflows.
settings	User preferences, legal references, app configuration flags, and future feature toggles.	Useful boundary for future premium and localization settings.

5. API design conventions

Area	Recommendation
Versioning	Prefix endpoints with /api/v1 and keep future versions isolated by router composition.
Resource naming	Use plural nouns for resources and stable, predictable URL patterns.
Validation	Validate every request with explicit Pydantic schemas; do not pass raw dicts through services.
Response shape	Return structured response models with pagination metadata where appropriate.
Errors	Map validation, auth, domain, and integration failures into normalized error payloads.
Idempotency	Use idempotency-conscious patterns for OTP send, retryable jobs, and external callback processing.
Pagination	Prefer cursor or stable page/limit conventions for list endpoints from the beginning.
Authorization	Centralize permission checks in dependencies and service-level guards rather than scattered inline conditions.
Observability	Attach request IDs and structured logs to every request lifecycle.
Docs	Keep OpenAPI descriptions meaningful so AI tools and developers can use the API spec as source documentation.

5.1 Suggested initial endpoint groups

Endpoint Group	Examples
Auth	POST /auth/request-otp, POST /auth/verify-otp, POST /auth/refresh, POST /auth/logout
Users	GET /users/me, PATCH /users/me
Pets	GET /pets, POST /pets, GET /pets/{id}, PATCH /pets/{id}, DELETE /pets/{id}
Topics	GET /topics, GET /topics/{id}
Chat	POST /chat/sessions, GET /chat/sessions/{id}, POST /chat/sessions/{id}/messages
History	GET /history/sessions, PATCH /history/sessions/{id}/archive
Recommendations	GET /recommendations, POST /recommendations/{id}/feedback
Notifications	GET /notifications, PATCH /notifications/{id}/read
Settings	GET /settings/me, PATCH /settings/me

6. Authentication and security approach

The backend should be designed so authentication implementation can change without rewriting downstream business logic. A stable auth service boundary should support either AWS Cognito-backed flows or a custom OTP + JWT implementation.

- Keep OTP send and OTP verification inside a dedicated auth service rather than directly in routers.
- Store access and refresh token policies centrally in security configuration.
- Use secure hashing where secrets or challenge data must be persisted.
- Apply rate limiting for OTP requests, login attempts, and chat abuse protection through Redis-backed throttling.
- Treat current user resolution as a reusable dependency for all protected endpoints.
- Store only the minimum necessary auth-related user data in the local application database when using external identity services.

7. Database and persistence strategy

Concern	Recommended approach
Transaction model	Use one request-scoped database session with explicit commit boundaries controlled in the service layer.
Repository pattern	Use repositories for reusable persistence logic, list/detail queries, and aggregate operations.
Read vs write queries	Keep heavy read formatting or joins out of routes; move them into repository or read-model helpers.
Migrations	Every schema change must ship with an Alembic migration and rollback-aware review.
Seeding	Use explicit scripts for topics, AI personalities, brands, products, and other controlled reference data.
Indexes	Plan indexes for phone numbers, session lookups, pet ownership, recommendation history, and recent activity sorting.
Soft deletion	Use soft delete only where product requirements justify retention; do not add it by default everywhere.
Enums	Centralize domain enums for statuses, sender types, recommendation states, and notification states.
Auditability	Keep created_at and updated_at fields standard across entities; add event tables later only if required.

8. Background jobs and async workloads

Workload Type	Why it should be async in V1
Push notifications	Non-blocking delivery and retry-friendly processing.
Chat title and summary generation	Can be deferred after the main chat response is saved.
Recommendation enrichment	Supports enrichment or fallback logic without slowing chat response time unnecessarily.
Analytics/event fan-out	Allows non-critical event recording without blocking user requests.
Media processing	Future-safe for image optimization or exported file generation.
Maintenance routines	Cleanup, cache invalidation, and scheduled housekeeping do not belong in synchronous API handlers.

- Use Celery with Redis as the first V1 task system; keep task payloads explicit and small.
- Prefer enqueue-after-commit patterns so tasks never run on data that failed to persist.
- Make tasks idempotent where retries are possible.
- Separate user-facing latency from internal enrichment whenever practical.

9. Redis usage strategy

Use Case	Redis role in RoBe V1
Rate limiting	Throttle OTP, login, and chat abuse patterns.
Task broker	Support Celery job dispatch and worker coordination.
Short-lived session helpers	Store OTP attempt state, verification windows, or ephemeral auth challenges.
Cache	Cache topic lists, personality lists, product lookups, and frequently accessed reference data.
Temporary chat helpers	Store transient typing, lock, or deduplication aids if needed.
Feature toggles / ops flags	Optional lightweight runtime controls for temporary operational switches.

10. AWS integration boundaries

Integration Area	Recommended backend boundary
S3 / CloudFront	Wrap upload URL generation, object key rules, and asset path building in a storage integration service.
SNS / Pinpoint / SMS provider	Hide OTP delivery implementation behind a notification or auth integration interface.
FCM / APNs bridge	Expose push dispatch through a notification delivery service, not directly in business modules.
CloudWatch / telemetry	Centralize structured logging, request correlation IDs, and operational metrics.
Secrets and environment config	Use strongly typed settings and environment-specific configuration loading.
AI provider calls	Create a dedicated AI integration boundary so prompt contracts, retries, and provider switching remain controlled.

11. Configuration and environment management

- Use one typed settings class as the single source of truth for environment variables.
- Separate local, staging, and production configuration clearly and avoid hidden fallback behavior.
- Keep sensitive values out of source control and provide a clean .env.example for onboarding.
- Document required environment keys for database, Redis, JWT/auth, storage, AI integrations, and telemetry.
- Support feature flags for experimental AI behavior, optional recommendation logic, and non-production tooling.

12. Observability and error handling

Area	Guideline
Logging	Use structured logs with request IDs, user context where appropriate, and task correlation metadata.
Exceptions	Define application exceptions and integration exceptions separately.
Monitoring	Track API health, latency, task failures, retry spikes, DB connectivity issues, and cache errors.
Sentry integration	Capture unhandled exceptions, route context, task context, and release versions.
Health checks	Expose readiness and liveness endpoints for deployment orchestration and monitoring.
Audit visibility	Log important admin-sensitive actions and operational anomalies deliberately, not noisily.

13. Testing strategy for V1

Test Type	Scope
Unit tests	Services, repositories, validators, mappers, permissions, and utility helpers.
Integration tests	Database-backed workflows such as onboarding, pet creation, chat initiation, and recommendation persistence.
API tests	Route-level tests for auth, validation, pagination, authorization, and common failure modes.
Task tests	Celery task behavior, retry cases, and post-commit assumptions.
Contract checks	Schema-level checks for API request and response contracts used by the frontend.
Smoke tests	Pre-release verification of app startup dependencies, DB migrations, worker startup, and core endpoints.

- Use pytest as the standard test runner.
- Prefer factories and fixtures over brittle copied setup blocks.
- Keep mock usage focused; integration tests should touch the real persistence layer where practical.
- Cover the chat module more heavily than low-risk static modules because it is the product core.

14. CI/CD and deployment workflow

Pipeline Stage	Expected checks
Static quality	Formatting, linting, import sorting, and type-aware checks.
Test execution	Unit and integration tests with controlled environment setup.
Build	Docker image build for API and worker containers.
Migration verification	Ensure database migrations apply cleanly before deployment.
Publish	Push images to registry and update ECS services.
Post-deploy validation	Run health checks and basic endpoint smoke verification.

15. Coding standards for AI-assisted backend development

- Every feature must have a predictable home in routers, schemas, services, repositories, and tasks.
- Do not place direct ORM queries inside route handlers.
- Do not call external services directly from models or schemas.
- Keep service methods focused on one use case and name them with clear verbs.
- Create explicit request and response schema classes instead of passing raw primitives through feature boundaries.
- Document non-obvious business rules close to the code module so AI tools can extend them safely.
- Use central constants and enums for status values rather than repeating strings across the codebase.
- Prefer small, testable functions over long orchestration blocks with mixed concerns.

16. Recommended initial implementation sequence

Phase	Main outcome
Phase 1	Project bootstrap: app wiring, settings, DB session, auth skeleton, health endpoints, Docker, and local development setup.
Phase 2	Core data model: users, pets, topics, personalities, chat sessions, messages, products, and recommendations.
Phase 3	Feature APIs: auth, pets, topics, chat session creation, message send, history list, and recommendation read flows.
Phase 4	Async and integrations: Celery, Redis, notification stubs, AI provider integration, summary generation, and caching.
Phase 5	Operational hardening: logging, Sentry, metrics, permissions, throttling, tests, and CI/CD deployment workflow.

17. Final recommendation

For RoBe V1, the backend should be built as a typed, modular FastAPI monolith using PostgreSQL, Redis, Celery, Docker, and AWS-managed infrastructure. This approach is strong enough for a real production launch, simple enough for a small team, and structured enough to serve as a high-value source document for AI-assisted development workflows.

- Faster onboarding for both developers and AI coding assistants.
- Clear boundaries between API, services, repositories, tasks, and integrations.
- A stable foundation for chat, pet context, recommendations, and future premium capabilities.
- Lower risk of messy code growth during the first version of the product.